

Atividade: Atividade Mini E-commerce Distribuído

Aluna: Amanda Montarroios de Oliveira

Como a comunicação entre os microsserviços foi implementada?

A comunicação foi feita via HTTP/REST. O gateway recebe todas as requisições e redireciona para o serviço certo com base na rota — /users vai pro serviço de usuários, /products pro de produtos e assim por diante. Internamente ele usa a biblioteca httpx do Python pra fazer essas chamadas de forma assíncrona.

Pra proteger a comunicação interna, configurei TLS com certificado autoassinado usando OpenSSL. Não é o ideal pra produção, mas cumpre o requisito de cifrar o tráfego entre os serviços. O token JWT é repassado direto no header Authorization sem modificação, então cada serviço valida ele por conta própria.

Qual estratégia de consistência foi adotada na replicação? Forte ou eventual? Por quê?

Tentei implementar consistência forte: toda vez que um produto é criado, o gateway escreve na primária (:5002) e logo em seguida manda um POST pra /products/_sync na réplica (:5012) antes de responder ao cliente. Se os dois confirmarem, a escrita é considerada bem-sucedida.

O problema é que se a réplica tiver offline na hora da escrita, o sistema ignora o erro e continua funcionando só com a primária — o que tecnicamente vira consistência eventual. Não implementei nenhum mecanismo de replay pra sincronizar a réplica quando ela voltar, então ela vai ficar desatualizada até a próxima escrita. Em produção isso seria um problema sério, mas pra essa atividade funcionou bem o suficiente.

O que acontece com o sistema se o Serviço de Pedidos cair?

Os outros serviços continuam funcionando normalmente — usuários ainda conseguem fazer login e produtos ainda podem ser consultados e criados. O gateway detecta que o serviço de pedidos caiu pelo heartbeat (que roda a cada 5 segundos) e começa a retornar 503 pra qualquer requisição que bata em /orders.

Testei isso derrubando o processo do orders manualmente e esperando uns 10 segundos. O log do gateway mostrou o timestamp da falha e o dashboard ficou vermelho pra esse serviço. Quando subi de novo, o heartbeat detectou a recuperação no próximo ciclo e voltou ao normal.

Como o JWT garante que um usuário comum não consiga criar produtos?

Na hora do login o serviço de usuários gera um JWT com os campos userId, email, role e exp. O role pode ser "user" ou "admin" dependendo do que foi cadastrado.

No serviço de produtos, o endpoint de criação passa pela função verify_admin antes de qualquer coisa. Essa função decodifica o token, verifica a assinatura com a chave secreta e

checa se o role é "admin". Se for "user", já retorna 403 na hora. Como o token é assinado, o cliente não consegue alterar o campo role sem a assinatura quebrar — então não tem como burlar isso sem conhecer a chave secreta do servidor.

Quais limitações a sua implementação possui em relação a um sistema real de produção?

Tem várias coisas que eu simplifiquei bastante:

Banco de dados em JSON: Funciona pra demonstração, mas não tem nenhum controle de concorrência. Se duas requisições tentarem escrever ao mesmo tempo no mesmo arquivo, uma pode sobrescrever a outra. Em produção usaria PostgreSQL ou MongoDB.

Replicação sem sincronização retroativa: Se a réplica ficar offline e voltar, ela não recupera os dados que perdeu. Precisaria de algum tipo de log de operações pra resolver isso direito.

Certificado autoassinado: O navegador e o httpx reclamam dele (por isso usei verify=False no gateway). Em produção usaria Let's Encrypt ou um certificado de uma CA real.

Sem rate limiting: O gateway aceita qualquer volume de requisições sem nenhuma proteção. Num ambiente real isso seria um problema óbvio.

JWT sem revogação: Se um usuário for desativado, o token dele ainda funciona até expirar (8 horas). Não implementei nenhuma blacklist de tokens.